Compte-rendu — Devoxx France 2026

Par : Usama OSMAN MOHAMED **Version** : 1.0 **Date** : 27 avril 2026
Dernière mise à jour : [27/04/2026]

Historique des versions

- **v1.0** (27/04) : Version Initiale du Compte-Rendu

Lieu : Paris **Dates** : 22-24 avril 2026 **Site officiel** : https://www.devoxx.fr/

Introduction

259 sessions. 307 intervenants. 10 thématiques. Devoxx France 2026 confirme son statut de rendez-vous incontournable de la tech francophone, avec une édition d'une densité rare couvrant l'IA agentique, la sécurité, les fondamentaux d'architecture, l'observabilité, le scheduling cloud, la cryptographie post-quantique et l'impact sociétal du numérique.

Devoxx France 2026 s'est ouvert sur une métaphore qui aura traversé toute l'édition — celle des échecs, racontée par Laurent Fressinet : trente ans après Deep Blue, ce n'est pas la machine qui a remplacé le joueur, c'est le joueur qui a appris à composer avec elle. Cette idée d'**humain augmenté plutôt que remplacé** résonne avec l'ensemble des trois jours.

Car si l'IA agentique est désormais sortie de l'exploration pour entrer en industrialisation — Doctolib affichant 100 % d'adoption, WeScale promouvant le spec-driven development, le standard `AGENTS.md` qui se généralise — la principale leçon de cette édition est paradoxale : **plus l'IA produit vite, plus l'humain doit être rigoureux**. Le contexte se discipline, les specs se versionnent, les workflows se structurent. Le *prompt engineering* devient *spec engineering*.

Cette exigence de rigueur traverse les autres grands thèmes de l'édition :

- un retour appuyé aux **fondamentaux de la conception** — dette de conception, architecture hexagonale, refactoring incrémental ;
- une attention renouvelée aux **fondations invisibles** — observabilité (OpenTelemetry), maîtrise du temps (NTP/PTP), scheduling Kubernetes ;
- une **maturité DevSecOps** assumée, du *shift-left* jusqu'à l'anticipation post-quantique ;
- une prise de conscience plus large de l'**impact sociétal** du numérique, en particulier sur les enfants.

Le fil rouge tient en une phrase : **l'augmentation n'est tenable que si la discipline suit**.

Mercredi 22 avril

9h30 — Keynote : Les échecs à l'ère de l'intelligence artificielle

Intervenant : Laurent Fressinet — Grand Maître International, double champion de France (2010, 2014), membre de l'équipe de préparation de Magnus Carlsen de 2013 à 2022 (5 titres de champion du monde),

ancien directeur France de chess24. Son positionnement est unique : il a traversé l'ère pré-informatique, l'arrivée des moteurs d'analyse, puis la révolution des IA modernes.

Contexte : les échecs comme laboratoire de l'IA

Les échecs ont longtemps symbolisé le raisonnement et la créativité humains. Ils sont aujourd'hui devenus un terrain d'expérimentation privilégié pour la collaboration homme-machine. Le moment charnière : la défaite de Kasparov face à Deep Blue en 1997, après que celui-ci avait affirmé ne jamais pouvoir être battu par une machine. Cet événement marque la fin du mythe de la supériorité humaine dans ce domaine.

Trois grandes transformations apportées par l'IA

1. **Préparation des joueurs** : l'analyse humaine et les livres d'ouverture ont laissé place à une utilisation massive de moteurs (Stockfish, AlphaZero…), permettant une exploration de lignes impossibles à calculer humainement. Résultat : hausse globale du niveau.
2. **Compréhension stratégique** : les IA ont introduit des concepts contre-intuitifs — certaines positions jugées mauvaises par les humains sont en réalité jouables. Sacrifices à long terme, jeu positionnel « non humain » : la vision du jeu est devenue plus dynamique et moins dogmatique.
3. **Esthétique du jeu** : contrairement aux craintes initiales, l'IA n'a pas tué la créativité. Elle a révélé de nouvelles formes de beauté — un style plus fluide, plus audacieux, parfois étrange, mais extrêmement efficace.

L'humain augmenté, pas remplacé

Point central de la keynote : l'IA n'a pas remplacé les joueurs, elle les a *augmentés*. Les joueurs utilisent aujourd'hui les IA comme partenaires d'entraînement, pour accélérer leur apprentissage et explorer de nouvelles idées. Les échecs illustrent ainsi comment l'humain s'adapte à une intelligence supérieure, apprend à collaborer avec une machine, et redéfinit sa propre créativité dans ce nouveau contexte.

> *La vraie révolution n'est pas la domination de la machine, mais la co-création entre humain et intelligence artificielle.*

10h30 — La dette de conception

Présenté par **Julien Topçu** et **Josian Chevalier** (Shodo). Un talk sur un problème courant : à force de corrections successives sans recul, le modèle métier d'une application se fragilise et devient incohérent. C'est ce qu'on appelle la **dette de conception** — plus insidieuse que la dette technique, car elle touche la façon dont on représente le métier dans le code. La session est portée par une mise en scène humoristique parodiant Le Seigneur des Anneaux et Star Wars pour illustrer les travers de la centralisation à outrance.

Le problème en une image : les orateurs utilisent le mot-valise **« cuichette »** (cuillère + fourchette) — et l'**effet Spork** — pour décrire ce qui arrive quand deux concepts distincts sont fusionnés dans un même objet. Le résultat n'est plus vraiment ni l'un ni l'autre, et chaque évolution devient un casse-tête. C'est la même logique que « le modèle unique pour les gouverner tous » : vouloir centraliser mène inexorablement à la **Grande Boule de Boue** (*Big Ball of Mud*).

Deux stades de dégradation à distinguer :

- **Entropie de modèle** : degré de désordre croissant dans le modèle — plus elle est élevée, plus la charge cognitive des développeurs augmente et plus les risques de régression s'accumulent
- **Sclérose du modèle** : stade terminal où le modèle est devenu si rigide qu'aucune modification ne peut être faite sans provoquer des erreurs majeures — le système se fige

Messages clés :

- Les **microservices sont un modèle de déploiement, pas un modèle de conception** — découper un système en petits services ne résout rien si le modèle métier est mal pensé ; on obtient une **boule de boue distribuée**, où chaque évolution nécessite des déploiements synchronisés entre équipes
- Une haute entropie entraîne un code spaghetti : moins de cohérence, des résultats imprévisibles
- L'organisation finit toujours par payer la dette de conception, que ce soit en bugs, en lenteur de livraison ou en incapacité à faire évoluer le produit

Comment détecter la dette de conception ? Les heuristiques de tension

Le talk propose une grille de lecture organisée en deux familles — **couplage intrinsèque** (au sein d'un même modèle) et **couplage extrinsèque** (entre modèles, équipes et organisation).

Couplage intrinsèque — tensions internes :

- **Effet Spork** : un modèle censé remplir deux rôles orthogonaux (ex. recherche et réservation) devient médiocre pour les deux
- **Infiltration d'état** : un état externe s'immisce dans un concept (ex. ajouter un booléen `isSelected` dans un objet `Fair`), brisant la cohérence structurelle
- **Tension linguistique** : le langage ubiquitaire (*Ubiquitous Language*) ne reflète pas la complexité réelle — des étapes clés du cycle de vie sont masquées derrière un vocabulaire trop pauvre
- Des booléens s'accumulent (`draft`, `validated`, `published`) là où il faudrait un vrai cycle de vie explicite
- Un concept simple à exprimer nécessite un code disproportionnellement complexe — signe de complexité accidentelle
- On transporte des informations inutiles « au cas où » un autre module en aurait besoin

Couplage extrinsèque — tensions sociotechniques :

- **Modèle BAPO** : les tensions se propagent entre le Business, l'Architecture, les Processus et l'Organisation — une mauvaise architecture force des négociations constantes sur les contrats d'API ou le « piratage » du backlog d'une autre équipe
- **Fuites de logique métier** : les règles de gestion sont dispersées entre services, rendant la maintenance impossible
- Le système empêche l'évolution du business (ex. impossible d'ouvrir un second entrepôt car le modèle ne gère pas le multi-site)
- Comprendre son propre système nécessite de comprendre celui de l'équipe voisine
- Les releases doivent être synchronisées entre équipes, le backlog de l'une bloque l'autre

Ces tensions sociotechniques ont leurs signaux organisationnels propres : réunions interminables sur les contrats d'API, déploiements à risque, backlogs bloqués entre équipes. Ce ne sont pas des fatalités — ce sont des **heuristiques** qui indiquent qu'un modèle est devenu obsolète.

La solution : la Mitose de Modèle

Pour sortir de l'impasse, le talk propose la **Mitose de Modèle** : séparer progressivement un modèle trop large en plusieurs contextes délimités (*Bounded Contexts*). La transition s'appuie sur un **noyau partagé** (*Shared Kernel*) temporaire, qui rétrécit à mesure que les nouveaux modèles gagnent en autonomie — évitant ainsi le big bang de la réécriture totale.

À retenir : les tensions de modèle sont des **signaux d'alerte précieux** — les identifier tôt permet d'agir avant que l'entropie ne mène à la sclérose. La Mitose de Modèle offre un chemin incrémental hors de la Grande Boule de Boue.

11h30 — Une belle leçon d'architecture logicielle (*Another World*)

Orateur : Olivier Poncet (Weyland-Yutani). Rétrospective sur *Another World* (1991), jeu d'aventure cinématique culte conçu par Éric Chahi seul (à l'exception de la musique), alors âgé de 22 ans. Le talk décortique l'architecture logicielle du jeu et montre comment des choix de conception visionnaires ont permis portage et longévité — jusqu'à une adaptation WebAssembly moderne réalisée par l'orateur lui-même.

Minimalisme et prouesses techniques

- Exécutable de 24 Ko, assets de 1,2 Mo : tout le jeu tient sur une seule disquette
- Développé sur Amiga 500 ; outils de dessin créés en BASIC par Chahi lui-même
- Rotoscopie pour certaines animations — Chahi filmait des mouvements réels, puis dessinait des polygones par-dessus, pour un réalisme inédit à l'époque

Le cœur du système : une Machine Virtuelle sur mesure

Le secret de la portabilité (le jeu a été porté sur plus d'une dizaine de plateformes) : plutôt que de cibler directement le matériel, Chahi a conçu une VM dédiée. Porter le jeu se résumait à réécrire l'interpréteur de la VM pour la nouvelle machine — le code de jeu, lui, ne changeait pas.

Architecture de la VM : type Harvard, big-endian, **64 threads coopératifs** pour animer personnages, ennemis et décors simultanément, **256 registres** de 16 bits portant l'intégralité de la logique de jeu, 1 pile. Tout est piloté par les ressources (vidéo, audio, inputs).

Moteur graphique : polygones vectoriels plutôt que sprites

Là où les jeux de l'époque s'appuyaient sur des images point par point, *Another World* utilise des polygones 2D — animations fluides et cinématiques pour un coût en données minimal. La gestion des couleurs recourt à des astuces (« OU inclusif ») pour simuler la transparence avec seulement 16 couleurs.

Ressources et audio

- Compression **ByteKiller** : fenêtre glissante, fonctionne à rebours, très économe en mémoire vive
- Audio : 4 pistes PCM gérées en temps réel (sons + musique), pilotées comme toutes les ressources par le système de VM

Conclusion

Cette architecture — VM indépendante du matériel, bytecode dédié, multitâche coopératif — fait d'*Another World* une leçon d'ingénierie moderne avant l'heure. Olivier Poncet en a lui-même produit un portage WebAssembly, preuve que les choix de conception d'un développeur seul en 1991 tiennent encore debout trente-cinq ans plus tard.

13h30 — Comment l'IA a transformé la création graphique de Devoxx France

Orateurs : Nicolas Martignole (Organisateur de Devoxx France, Principal Engineer chez Back Market) et **Stéphane Itschner** (DA / Graphiste chez ALT Codes).

Depuis trois ans, les vidéos d'introduction et l'identité visuelle de Devoxx France ne passent pas inaperçues. Cette session en dévoile les coulisses : une collaboration à deux voix, entre un graphiste et un organisateur, sur comment les outils d'IA génératifs ont bouleversé le processus de création visuelle — avec, en filigrane, un message à destination des développeurs : ce bouleversement n'est qu'une préfiguration de ce qui nous attend.

Rétrospective visuelle — Les grandes enjambées techniques (2023–2026)

La session s'ouvre sur une timeline des évolutions techniques : chaque année a apporté un saut qualitatif visible dans les rendus (gap image en 2023–2024, gap vidéo en 2024–2025, puis Nano Gamma en 2026). Des outils comme **Kling** et **Weavy** sont cités parmi les plus avancés explorés pour la génération vidéo.

Avant vs. Maintenant — « C'était pas forcément mieux avant ! »

Le cœur de la session : la comparaison des workflows créatifs.

Avant (7 étapes séquentielles) : Brief → Storyboard → Validation → Premiers tests rendus → Adaptation → Production → Livrables. Un processus long, linéaire, avec des allers-retours coûteux entre chaque étape.

Maintenant (4 étapes itératives) :

1. On pense à un truc qui nous fait marrer — on réfléchit globalement
2. On teste un visuel
3. On lance les rendus
4. On adapte le son, la narration, le storyboard en fonction des meilleurs rendus — et parfois on change de cap en cours de route

Le workflow est devenu **fondamentalement itératif et exploratoire** : on ne valide plus un storyboard avant de produire, on laisse les rendus guider la narration. La logique s'est inversée.

Messages clés

- Évolution de l'outillage : du PC vers le cloud
- Workflow de génération d'images **beaucoup plus itératif** : plus de tests, plus d'essais
- *« You'll never be the GOAT again »* : on est spécialiste le temps d'un projet
- Approche **die and retry**
- Volume de production en forte hausse
- La **« voie du ninja »** (10 ans pour devenir expert) est révolue
- Ce que vivent les créatifs aujourd'hui **préfigure ce que vivront les développeurs** demain

14h30 — De 0 à des milliards de traces : l'observabilité chez Winamax

Contexte : ~500 employés à Paris, stack full JS, React côté front, très couplé à AWS, **700+ microservices**.

Problème de départ : trop de logs, trop de métriques.

Solution : passage au standard **OpenTelemetry**, organisé en 4 piliers :

1. **Production des traces** → production de spans
2. **Collecte** : moments d'intensité (schedule rapide), pattern sidecar évité (trop complexe), choix d'un cluster gateway
3. **Stockage** : simple, rapide, résistant à la charge, compatible OpenTelemetry → observabilité self-hosted avec **Quickwit sur S3**
4. **Recherche** : pilotée par 2 questions — coût ressources et volumétrie

15h40 — Tic-Tac... ! Maîtrisez le temps et NTP avant qu'il ne soit trop tard

Présenté par **David Santiago** et **Cynthia Treger** (Microsoft). Un talk mêlant histoire, anecdotes techniques et bonnes pratiques — de l'ombre des cadrans solaires aux horloges atomiques des satellites GPS.

Notions clés

- GMT ≠ UTC ≠ TAI
- UTC = compromis entre heure atomique et zénith solaire
- La rotation de la Terre engendre des **leap seconds**, ajoutées par l'Observatoire de Paris avec 6 mois d'avance, le 30 juin ou le 31 décembre

Incidents historiques liés aux leap seconds

- 2012 : Reddit, Cloudflare, LinkedIn
- 2015 : DNS global
- 2017 : Cloudflare

À noter : **arrêt des leap seconds prévu à partir de 2035**, avant qu'un saut brutal soit éventuellement décidé.

NTPd vs Chrony — choisir le bon moteur

Le talk consacre un comparatif détaillé entre les deux implémentations NTP :

Critère	NTPd (legacy)	Chrony (modern)
Sync Speed	Lent (peut prendre des heures)	Très rapide (secondes à minutes)
Environnement	Serveurs physiques, "always-on"	Cloud, VMs, Conteneurs
Connectivité	Connexion stable et permanente requise	Gère les liens intermittents
Précision	Millisecondes (ms)	Microsecondes (µs)
Empreinte	Usage mémoire standard	Plus léger en RAM et CPU
Algorithme	Conservateur, lisse la dérive lentement	Adaptatif, réagit vite aux décalages

Conclusion : **Chrony est le choix recommandé** pour tout environnement cloud ou virtualisé.

Bonnes pratiques — Configuration Chrony (cloud-init)

Paramètres clés à maîtriser dans `/etc/chrony/chrony.conf` :

- `driftfile /var/lib/chrony/chrony.drift` — enregistre le taux de dérive de l'horloge système ; indispensable pour maintenir la précision après un reboot

- `maxupdateskew 100.0` — définit la déviation de fréquence maximale autorisée (100 ppm est une valeur conservatrice standard)
- `makestep 1.0 -1` — si le décalage dépasse 1 seconde à n'importe quel moment (`-1`), Chrony corrige l'horloge **brutalement** (*step*) plutôt que progressivement (*slewing*) ; à utiliser avec discernement

Bonnes pratiques — Horloge wall clock vs horloge monotonique

L'erreur classique : confondre *quelle heure il est* avec *combien de temps ça a pris*.

- **Wall clock** : l'heure absolue du monde réel — peut reculer (NTP resync, leap second, DST). À utiliser pour les **logs, enregistrements en base, timestamps métier**.
- **Monotonic clock** : temps écoulé depuis un point fixe arbitraire (boot), haute résolution, **ne recule jamais**. À utiliser pour les **mesures de durée, timeouts, rate limiting, benchmarks**.

Quand la milliseconde ne suffit plus — PTP

NTP couvre **95 % des cas d'usage** avec une précision à la milliseconde sur Internet. Pour les 5 % restants — bases de données distribuées, systèmes haute fréquence, télécom — une milliseconde est déjà trop imprécise.

PTP — Precision Time Protocol : même objectif que NTP (garder les horloges synchronisées) mais approche radicalement différente :

- **LAN** au lieu d'Internet
- **Timestamping hardware** (par la NIC au moment où le bit touche le câble) au lieu de software
- Précision **µs/ns** au lieu de ms

PTP Deep Dive — l'horloge passe hardware

Trois mécanismes clés :

1. **Hardware timestamping** : le timestamp est apposé par la NIC au moment précis où le bit touche le fil — le jitter noyau et OS est complètement éliminé
2. **PHC — PTP Hardware Clock** : l'horloge maître de l'hôte est exposée à la VM comme un périphérique local : `/dev/ptp_hyperv` (Azure) ou `/dev/ptp0` (bare metal)
3. **Stratum 0 local** : on se branche directement sur la source de vérité locale — plus d'Internet, plus de sauts réseau

Résultat visible avec `chronyc sources` : la source réseau disparaît, remplacée par `PHC` avec une précision de **−24 µs à −26 µs ± 6 569 ns**.

Bonnes pratiques — résumé

- **4 sources NTP** pour disposer d'un quorum et isoler un éventuel serveur menteur
- Préférer **Chrony** à NTPd en environnement cloud/VM
- Configurer `driftfile`, `maxupdateskew` et `makestep` consciencieusement
- Utiliser l'**horloge monotonique** pour toute mesure de durée dans le code
- Pour des besoins sub-milliseconde : basculer sur **PTP** avec timestamping hardware

17h — Stop à la dette documentaire : Industrialisez vos ADRs et READMEs avec Spec-kit (WeScale)

Présenté par **Alexandre Guillemot** et **Axel Chauvin** (WeScale). Un Tools-in-Action en live, sans slides, partant d'un dossier vide pour construire le socle documentaire d'un projet. La thèse centrale : la documentation n'est pas un luxe, c'est l'assurance-vie de l'agilité — sans elle, chaque modification devient un risque et l'onboarding un calvaire.

GitHub Spec-kit en pratique :

- **Standardisation** : bootstrap d'une structure documentaire cohérente (Clean Arch + `docs/`) pilotée par le code
- **Discipline automatisée** : la CI devient le garant du patrimoine technique — blocage de PR sans ADR, validation de structure
- **DevEx** : réduction de la charge cognitive en automatisant la conformité

Promotion du **Spec-Driven Development** :

- La spec vit dans le repo, la CI la lit
- *Constitution* : les règles humaines
- *Spec IT* : 5 phases (`spec.md`) avec Claude / IA
- **Règle d'or** : toute modification est accompagnée d'une doc générée par IA
- CI/CD : l'IA vérifie la cohérence entre le commit et la doc associée
- Mention de **Kiro** (GovCloud)

GitHub Spec-kit — éléments de contexte

- Projet open source porté par **GitHub** : les specs deviennent des artefacts *exécutables* qui génèrent directement des implémentations plutôt que de simples documents de guidance
- Flux piloté par des **slash commands** : `/specify` → `/plan` → `/tasks` → `/implement`, chacune correspondant à une phase du cycle de développement
- Workflow **define-and-apply** en deux phases : possibilité d'affiner les specs en cours d'implémentation
- Intégration avec les outils de ticketing (Jira, Linear, Azure DevOps, GitHub Projects)
- Agnostique quant à l'agent IA : compatible Claude Code, GitHub Copilot, Cursor, etc.

Jeudi 23 avril

> Beaucoup trop de monde partout cette journée.

Stands visités :

- **Docker** : sandbox pour agents LLM
- **Chainguard** (déjà en contact avec Samuel Ménard) : sécurité des conteneurs avec traçabilité

10h30 — Grandir avec le numérique : et si on apprenait aux enfants à s'en protéger ?

Oratrice : Estelle Landry — Product Manager avec plus de 10 ans d'expérience dans la conception de produits numériques à fort impact, chez **Pix** (groupement d'intérêt public au service des citoyens français dans l'apprentissage du numérique).

Talk non-technique mais très marquant surtout pour les parents.

Le constat alarmant sur les usages numériques

La conférence débute par le témoignage fictif mais réaliste de **Maya**, 10 ans, victime de cyberharcèlement par ses camarades (« t'es nulle, t'es moche »), avec peur de retourner à l'école. Caractéristiques du phénomène : **rapide**, **immuable**, **omniprésent**.

Chiffres marquants :

- **99 %** des enfants de 11 ans se connectent quotidiennement au web
- Temps d'écran moyen entre 13 et 17 ans : **6h35 par jour**
- Âge moyen du premier smartphone : **9 ans et 9 mois**
- **90 %** des enfants exposés à des contenus choquants avant 15 ans
- **Pix Junior** : **7 millions d'utilisateurs**

Point critique : le mythe des *Digital Natives* est déconstruit. Les enfants savent « scroller », mais ils n'ont **pas les compétences pour se protéger** ou comprendre le fonctionnement des outils.

La mission de Pix Junior

Face à l'augmentation des cas de cyberharcèlement et au manque d'outils pour les enseignants, Pix Junior a été créé en 2022 pour les élèves de **CM1 et CM2**, avec trois objectifs :

- **Autonomie** : utilisable par les élèves seuls, pour pallier le manque de matériel ou de temps des enseignants
- **Évaluation** : permettre aux professeurs de suivre la progression réelle des compétences numériques
- **Engagement** : maintenir la motivation grâce à une approche ludique et bienveillante

Méthodologie de conception — Test & Learn

La démarche s'appuie sur une coconstruction rigoureuse : plus de **250 enfants** et **20 enseignants** rencontrés pour identifier les frictions.

Choix pédagogique clé sur le **sentiment d'auto-efficacité** (Bandura) : l'interface évite de mettre l'enfant en échec. Ce sont les *personnages de l'application* qui rencontrent des problèmes, et l'enfant doit les aider — ce qui réduit la charge émotionnelle de l'erreur.

Algorithme d'apprentissage adaptatif en trois niveaux :

1. **Validation** : vérifier ce que l'enfant sait déjà
2. **Entraînement / Didacticiel** : proposer des aides pédagogiques en cas d'échec, pour amener progressivement vers la maîtrise
3. **Défis** : pour éviter l'ennui des élèves les plus avancés

Le concept de **ZPD** (Zone Proximale de Développement) est central : l'outil cherche à maintenir l'élève dans la zone où il peut réussir avec un léger guidage, évitant ainsi l'ennui comme le découragement.

Résultats

- **82 %** des élèves ont progressé grâce au niveau d'entraînement
- **62 %** des élèves en grande difficulté ont réussi grâce aux didacticiels

Conseils aux bâtisseurs de la tech

- Prioriser la **valeur réelle** plutôt que la vélocité de développement

- Remplacer les roadmaps de fonctionnalités par des **roadmaps d'expérimentation**
- Travailler en **équipes pluridisciplinaires** (Tech + Experts métier) pour varier les angles de réflexion

La conférence se conclut par un appel à la responsabilité des créateurs de technologies : concevoir des outils qui **protègent les plus jeunes** plutôt que de les exposer. Un message adressé à tous — designers, devs, PM, CTO, citoyens.

11h30 — L'Agentic Coding, nouveau territoire du Platform Engineering (Doctolib)

Orateurs : Yankı Sesyılmaz (Staff SRE) et **Julien Tanay** (Senior Software Engineer) — équipe AI Scaling chez Doctolib.

Un talk de platform engineering avant d'être un talk sur l'IA : comment construire une plateforme IA collaborative pour 600 ingénieurs, au-delà du simple taux d'adoption. Sujets couverts : agentic coding, expérience développeur, architecture de plugins extensible, métriques pertinentes et échecs réels.

Le chiffre de départ

Doctolib (~600 ingénieurs) affiche une **adoption à 100 %** de l'IA :

- 2025 : phase d'exploration
- 2026 : adoption massive de **Claude Code**, passage en mode **agentic-first**

Mais 100 % d'adoption n'est pas la fin — c'est le début des vrais problèmes de plateforme.

Une task force de 2 pour 600

L'équipe AI Scaling qui pilote tout ça : **2 personnes dédiées** pour accompagner 600 profils tech. Une organisation légère qui impose de prioriser, standardiser et industrialiser plutôt que de traiter cas par cas.

La plateforme : architecture ouverte aux contributions

L'enjeu n'est pas « quel outil choisir » mais comment construire une plateforme extensible que les équipes métier peuvent enrichir elles-mêmes.

Pipeline d'industrialisation de l'outillage en 5 phases :

1. **Veille** : identification des outils et pratiques émergentes
2. **Filtre valeur** : est-ce que ça répond à un problème réel des équipes ?
3. **Revue sécurité** : validation avant tout déploiement
4. **Test avec AI champions** : montée en compétence par les utilisateurs avancés
5. **Standardisation** : intégration à la marketplace et aux plugins pré-installés

Architecture de la plateforme :

- **Marketplace Claude** : catalogue de plugins et skills disponibles pour toutes les équipes
- **Plugins pré-installés** : socle commun, sans friction au démarrage
- **Architecture extensible** : les équipes métier contribuent leurs propres plugins
- **Savoir ne pas construire** : exemple emblématique — leur outil interne « Claude hat » abandonné au profit de Claude Desktop d'Anthropic dès que celui-ci a couvert le besoin

Spec-Driven Development

Le *prompt engineering* devient du **spec engineering** :

- Specs en **Markdown uniquement**, lisibles humains et machines
- Le plan devient un **artefact versionné** dans le repo
- Framework retenu : **OpenSpec** (à ne pas confondre avec Speckit de WeScale) — choix délibéré pour garder un cadre léger et adaptable
- Investissement fort dans les pratiques de context engineering

Qui adopte le mieux l'agentic ?

Observation intéressante : les profils déjà habitués au *context switching* adoptent le mieux le mode agentic. Les **DevOps/SRE**, très transverses par nature, sont parmi les plus grands utilisateurs. Leviers pour réduire la friction : worktrees git, dashboards d'agents, réduction des blocages humains dans les workflows.

Remote agents — cas d'usage concrets

Les agents autonomes via **GitHub Actions** couvrent des use cases bien réels :

- Traitement du backlog de tickets
- Gestion d'incidents de production
- Migrations techniques
- Revue de code automatisée

Infrastructure sous-jacente : orchestrateur dédié, observabilité des agents, guardrails pour cadrer les comportements.

Métriques au-delà du taux d'adoption

Le talk insiste : le taux d'adoption est une métrique de départ, pas d'arrivée. Les équipes plateforme doivent suivre la qualité de l'usage, les patterns de context engineering, les contributions à la marketplace, et la réduction effective des frictions dans les workflows.

Ce que le talk ne résout pas — mais pose bien

Par honnêteté, les orateurs soulignent que l'adoption reste **inégale** en pratique, et que le talk pointe les bonnes questions à se poser plus qu'il ne dicte comment les résoudre. Un REX utile précisément parce qu'il n'élude pas les échecs.

13h30 — The Hive : coder un monolithe modulaire prêt pour les microservices

Orateurs : Thomas PIERRAIN et **Julien Topçu** (Shodo). Session de live coding partant d'un monolithe spaghetti legacy pour aboutir à un monolithe modulaire, avant de conclure sur l'extraction d'un module en microservice autonome. Le talk prolonge directement la réflexion ouverte le matin sur la dette de conception, en apportant cette fois la solution concrète.

Le constat de départ

Une décennie de microservices a montré les limites du découpage précipité : des microservices mal conçus se transforment en monolithe distribué, encore plus problématique que le monolithe spaghetti qu'ils étaient censés remplacer. Le **monolithe modulaire** s'impose aujourd'hui comme une alternative sérieuse — à condition de savoir le découper sans créer de modules trop fortement couplés.

Le Hive Pattern — principe

Le **Hive Pattern** (pattern « ruche ») combine l'**architecture hexagonale** et le **vertical slicing** pour construire un monolithe modulaire réellement prêt à être découpé en services.

Principe de l'architecture hexagonale : séparer la logique métier (le « domaine ») de tout le reste (base de données, API, interfaces). Le domaine ne connaît que ses propres règles et communique avec l'extérieur via des interfaces (« ports »), implémentées par des « adaptateurs ». Cela rend la logique métier indépendante des choix techniques.

Le Hive Pattern pousse cette idée plus loin :

- Chaque module correspond à un **domaine métier** (bounded context) et forme un hexagone autonome avec sa propre base de données, ses propres tests, ses propres adaptateurs
- Les modules communiquent entre eux **uniquement via des ports et adaptateurs**, comme s'ils étaient déjà des services séparés — même quand ils tournent dans la même application
- On peut ainsi démarrer avec un **monolithe modulaire** (une seule application, plusieurs modules bien découpés), puis extraire n'importe quel module en microservice indépendant **sans réécriture** : il suffit de remplacer l'adaptateur « en mémoire » par un adaptateur réseau

Démonstration live

La DeLorean de *Retour vers le futur* comme fil conducteur, avec 4 modules métier (DeLorean, circuit temporel, réacteur nucléaire, convecteur temporel). Le live coding part du monolithe spaghetti legacy, le restructure module par module selon le pattern Hive (vertical slicing in-process), puis extrait le réacteur nucléaire en microservice séparé — le code métier ne change pas, seul l'adaptateur de communication change (mémoire → réseau).

Ce qu'une archi hexagonale mal pensée peut produire

Point de mise en garde : une architecture hexagonale mal appliquée peut elle-même produire du spaghetti si les modules ne correspondent pas à de vrais domaines métier. Il faut impérativement séparer les fonctions selon leur rôle et s'assurer que chaque hexagone a une responsabilité cohérente.

À retenir : le Hive Pattern offre un chemin de modularisation progressif — on ne réécrit pas tout d'un coup, on prépare l'extraction dès la conception du monolithe.

14h30 — Secure all the things: From IaC to Kubernetes (DevSecOps)

Orateur : Romain Boulanger — Cloud and DevSecOps Architect @ Piguet Galland & Cie S.A. | Slides

Retour d'expérience sur 2 ans de transition DevSecOps chez Piguet Galland & Cie (banque privée), avec pour objectif de poser les fondations nécessaires au déploiement d'un nouvel e-banking — on-premise et cloud. La présentation s'organise autour d'une série de questions pratiques, de l'IaC jusqu'à la sécurisation réseau dans Kubernetes.

Comment déployer vers le cloud ? → IaC avec OpenTofu

L'infrastructure est décrite comme du code (IaC) pour garantir l'automatisation, l'auditabilité, la cohérence entre environnements et la réduction des erreurs humaines.

Choix notable : **OpenTofu** plutôt que Terraform — fork open source, sans restrictions de licence, compatible drop-in, avec un apport clé : le **chiffrement natif du state et du plan** (depuis v1.7.0) via de nombreux providers de clés (PBKDF2, AWS KMS, GCP KMS, External…).

Pipeline IaC sécurisé (shift left) :

- **Lint** : Format + **TFLint**
- **Security** : **Checkov** (mauvaises configurations), **Secret detection**, **KICS**
- **Deployments** : Plan → Apply par environnement (test, quality, production), uniquement depuis la branche principale
- Règle d'or : **la pipeline doit être le seul moyen de déployer** ; des rôles temporaires élevés sont fournis en cas de panne ou de troubleshooting

Les pipelines elles-mêmes sont mutualisées via les **GitLab CI/CD Components** : des composants versionnés et réutilisables que les projets importent au lieu de dupliquer du YAML.

Comment packager les applications ? → Container Build sécurisé

Pipeline de build standardisée en 4 étapes :

- **Hadolint** : lint du Dockerfile
- **Syft / Grype** : génération du Software Bill of Materials (SBOM) et analyse des dépendances
- **Kaniko** : build rootless, sans démon Docker
- **Trivy** : scan de vulnérabilités (CVE) sur l'image produite

Bonne pratique rappelée en bas de slide : "*Don't forget to scan your images at runtime too!*"

Le **multi-stage build** est également présenté comme levier de sécurité : seules les dépendances runtime se retrouvent dans l'image finale — pas les outils de build ni le code source, ce qui réduit la surface d'attaque et la taille de l'image.

Comment gérer les dépendances ? → Renovate Wizard

Renovate Bot automatise la mise à jour de toutes les dépendances (ansible, helmv3, dockerfile, gitlabci, terraform…) en créant des merge requests avec les changelogs détaillés — *Merge Requests as a Service*.

Comment déployer dans Kubernetes ? → Talos Linux + Cluster API + Argo CD

Le cluster tourne sur **Talos Linux** — distribution Linux minimaliste et immuable conçue exclusivement pour Kubernetes : pas de shell, pas de SSH, pas de gestionnaire de paquets. Toute interaction passe par une API sécurisée en mTLS (`talosctl`), ce qui réduit drastiquement la surface d'attaque et élimine la dérive de configuration.

Le provisionnement des clusters est piloté par **Cluster API (CAPI)** : on commence par monter un premier cluster Kubernetes (le cluster de management), sur lequel on installe les controllers CAPI. C'est ce cluster de management qui devient ensuite la tour de contrôle depuis laquelle on déploie et gère le cycle de vie de tous les autres clusters Talos (création, mise à jour, suppression), de façon entièrement déclarative.

Argo CD comme chef d'orchestre GitOps en mode pull :

- Git comme source de vérité unique
- Pas de credentials cluster stockés dans la pipeline

- Réconciliation en boucle automatique
- Interface de contrôle pour la visibilité des workloads et la gestion des permissions

Comment déployer les secrets ? → External Secret Operator + SOPS

La solution retenue pour la gestion des secrets est l'**External Secret Operator** (ESO), outil CNCF sandbox qui se connecte à de nombreux providers externes (HashiCorp Vault, Azure Key Vault, Delinea…). L'ESO surveille les `ExternalSecret` dans Kubernetes et crée les `Secret` correspondants en allant les récupérer dans le vault.

Le problème de l'œuf et la poule : l'ESO a besoin d'un secret (les credentials du vault) pour aller chercher des secrets. Comment commiter ce premier secret dans Git sans l'exposer en clair ?

Solution : **SOPS** (*Secrets OPerationS*), qui chiffre uniquement les valeurs sensibles dans les fichiers YAML — les clés restent lisibles, seules les valeurs sont chiffrées (`clientSecret: ENC[AES256_GCM,...]`). Le workflow de bootstrap : la pipeline récupère la clé SOPS, puis lance `helm install` pour déployer l'ESO avec ses credentials déchiffrés à la volée.

Comment sécuriser le déploiement ? → Policy as Code avec Kyverno

Kyverno : policy management Kubernetes-natif, sans dépendance externe, avec des politiques en YAML (pas de nouveau langage). Types de politiques disponibles : Validate, Mutate, Generate, Delete. Large catalogue communautaire de politiques prêtes à l'emploi.

Exemple montré en slide : interdiction du tag `:latest` sur les images de conteneurs (`disallow-latest-tag`).

Comment sécuriser le réseau ? → Istio Ambient + AuthorizationPolicy

Istio en mode Ambient : service mesh offrant observabilité, mTLS et fonctionnalités réseau — *sans modifier une seule ligne de code applicatif*. Architecture en deux composants dans le data plane : `ztunnel` (nœud) et `waypoint` (namespace), coordonnés par `istiod` dans le control plane. La visibilité du mesh est consultable via **Kiali**.

Limites des NetworkPolicy (slide dédiée) : filtrage basique L3/L4 (IP et ports uniquement), **pas de deny explicite** (il faut créer une politique vide), et surtout les **labels peuvent être modifiés sans redémarrer le pod** — une faille de sécurité.

AuthorizationPolicy — la réponse (sous-titre : "*Zero trust, here we come!*") :

- **Sécurité basée sur l'identité** : service accounts au lieu de labels mutables
- **Filtrage L7** : méthodes HTTP, chemins, headers, JWT claims
- **Actions ALLOW/DENY explicites** : politiques claires et non ambiguës

Conclusion

- Notre voyage n'est pas terminé — il fait à peine ses débuts
- L'automatisation et les outils ne sont que des moyens, pas une fin
- Le DevSecOps est avant tout un état d'esprit
- La sécurité est l'affaire de tous

> *(Note personnelle)* La surveillance runtime des comportements suspects dans Kubernetes (détection d'anomalies, processus inattendus, écriture dans des répertoires système…) est un sujet

> complémentaire non abordé dans ce talk. **Falco** est l'outil de référence de l'écosystème pour ce cas d'usage.

17h — Domptez vos agents : AGENTS.md et Context Engineering pour une IA déterministe

Orateur : Benoît Fontaine — Architecte Groupe chez Septeo.

La thèse centrale : là où la plupart des développeurs se contentent de prompts isolés, il faut aller bien au-delà du *prompt engineering* pour atteindre le **context engineering** — transformer ses assistants IA (Claude, Cursor…) en véritables partenaires d'ingénierie fiables, via la standardisation du contexte.

Benoît Fontaine a publié une série de trois articles sur le sujet :

- Pourquoi vos agents IA codent mal — le vrai problème, c'est le contexte
- Context Engineering — Partie 2
- Context Engineering — Partie 3

Ce que la recherche prouve — les limites des agents sans contexte

Trois benchmarks éclairants présentés en ouverture :

- **FeatureBench (74 % + 11 %)** : les agents ne *développent* pas de features — ils *fixent des bugs*. Performance de Claude mesurée sur SWE-bench + FeatureBench (évalué sur 34 apps).
- **SWE-ABS (1,6 abstraction)** : « passer les tests » ≠ « code correct » — le patch réel et un développement sain sont administrativement proches, mais les tests sont trop faibles pour attester l'absence d'erreur.
- **ContextBench** : le contexte fourni *en amont* bat systématiquement le contexte exploré dynamiquement par l'agent.

Constat résumé : seulement ~32 % du code produit par les agents est réellement utilisé et validé par les développeurs. La solution n'est pas de changer de modèle, mais de mieux structurer le contexte.

Le standard `AGENTS.md` — anatomie d'un fichier de configuration performant

`AGENTS.md` est le « cerveau » du projet : un fichier de configuration qui définit pour l'agent son rôle, ses contraintes et ses commandes exécutables.

Hiérarchie de chargement et fichiers par outil — « le bon contexte, au bon moment, pour le bon outil » :

- Niveau projet : `AGENTS.md` (**standard ouvert**), `CLAUDE.md`, `GERENS.md`
- Niveau répertoire : `CLAUDE.md` (contexte local affiné)
- Niveau sous-répertoire : `CLAUDE.md` (encore plus précis)

Contenu clé d'un `AGENTS.md` performant :

- **Définition de personas** : qui est l'agent, quel est son rôle dans ce projet
- **Limites (boundaries)** : ce que l'agent ne doit *jamais* faire — stopper les hallucinations
- **Commandes exécutables** : des instructions claires et actionnables
- **Règles d'architecture** : SOLID, conventions de nommage, standards du projet
- Moins de **200 lignes** — chaque ligne consomme du contexte à chaque message

Le workflow TDD/BDD assisté

Structurer des User Stories qui *forcent* l'agent à écrire les tests avant le code (cycle Red-Green-Refactor), garantissant la qualité et le respect des principes SOLID. L'agent ne peut avancer qu'en respectant la discipline de test imposée par le contexte.

Gestion de la mémoire — éviter le *context rot*

Le *context rot* (dégradation du contexte) est le principal ennemi sur les tâches longues. La réponse : le pattern **« une session, une tâche »**.

Trois phases, compaction entre chaque — le contexte ne dépasse jamais 60 % de la fenêtre :

1. **Session 1 — Recherche** : explorer la codebase, identifier les fichiers impactés, évaluer les risques → ~60 % contexte utilisé, puis compaction
2. **Session 2 — Plan** : décomposer en tâches, valider avec le développeur → ~60 % contexte, puis compaction
3. **Session 3 — Implémentation** : exécuter le plan, TDD, vérifier et committer → ~60 % contexte

Multi-sessions et sous-agents — pendant la Session 3, l'agent principal peut déléguer à des sous-agents spécialisés, chacun avec un contexte limité :

Sous-agent	Rôle	Outils
Architecte	Lecture seule	Obus
Implémenteur Rust	Code Rust	Sonnet, src
Implémenteur Flutter	Code Flutter	Sonnet, lib
Agent de tests	TDD	Sonnet, TDD tests

Résultat : 4 sous-agents × 200 k tokens = **800 k de contexte effectif** dans une fenêtre de 200 k.

La méthode en 5 points

1. **Créez votre `CLAUDE.md` / `AGENTS.md` dès le jour 1** — moins de 200 lignes, chaque ligne consomme du contexte à chaque message
2. **Un prompt = une tâche = une couche** — committez vos fichiers clés ; ne laissez pas l'agent terminer *in abstracto*
3. **Compactez à 60 % de la fenêtre** — ne dépassez jamais 80 % ; forcez la compaction à 60 % en précisant l'impact
4. **Documentez avant de `/release`** — l'historique git est votre mémoire ; plan-archivé et plan avant d'initialiser
5. **Appliquez ces ratios à vos systèmes MCI en production** — agent principal + `CLAUDE.md` revu + réduction automatique du contexte

> *Passez d'une IA qui « suggère du code » à une IA qui respecte vos standards d'architecture et vos exigences métier.*

17h50 — Claude Code tips

Titre complet : *ClaudeCode.proTips(30, minutes=30).run()* **Orateur** : Erwan Gereec (Doctolib)

30 minutes, 30 astuces. 1 mission : transformer la manière de travailler avec Claude Code. La session couvre prompting, refactoring, debug, tests, APIs, gestion du contexte et UI — pour faire de Claude Code un vrai copilote, bien au-delà d'un simple générateur de code.

Tip 1 — Utilisez l'installation native

Installer Claude Code via l'**installation native** (`npm install -g @anthropic-ai/claude-code`) plutôt que via un wrapper ou un gestionnaire de paquets tiers. Les mises à jour sont ainsi automatiques et silencieuses : Claude Code se met à jour de lui-même à chaque nouvelle version, sans manipulation supplémentaire. Passer par un outil tiers (Homebrew, etc.) casse ce mécanisme et oblige à mettre à jour manuellement.

Tip 2 — Customize ton Claude

- `/theme` pour choisir votre thème de couleur ; `/color yellow|green...` pour la couleur de l'invite
- `/statusline` pour décrire en langage naturel ce que vous voulez afficher dans la barre de statut (ex : `/statusline show active model and context usage in percent`) — ou configurez-la manuellement avec un script `.sh` intégré dans `~/.claude/settings.json`

Tip 3 — Adaptez votre affichage avec `/tui` et `/focus`

- `/tui` pour les échanges rapides : affichage léger, navigation clavier
- `/focus` pour un rendu épuré centré sur la tâche en cours — élimine le bruit visuel et mémorise la dernière fenêtre utilisée

Tip 4 — Nommez votre session avec `/rename`

- `/rename` nomme la session courante et affiche ce nom dans la barre Claude
- Sans cette commande, Claude génère un nom automatiquement à partir du contenu de la conversation
- Particulièrement utile dès que plusieurs sessions tournent en parallèle
- Bonne pratique : utiliser `/rename` dès le lancement de la session

Tip 6 — Le training facile avec `/powerup`

- Parcours interactif intégré : 10 leçons couvrant les fonctionnalités clés de Claude Code, chacune avec une explication et une mini-démo
- Sujets couverts : navigation dans la codebase avec `@`, modes de permission (`Shift+Tab`), `CLAUDE.md`, `/memory`, MCP, sub-agents…

Tip 7 — Accélérez votre saisie avec `!` et `@`

- `!` — **Exécuter une commande bash** : ex. `!git status` — exécute la commande et injecte le résultat dans le contexte ; tab-completion disponible depuis les commandes `!` précédentes
- `@` — **Référencer un fichier** : ex. `@src/auth/login.ts explique-moi cette fonction` — injecte directement le contenu du fichier dans le contexte, avec autocomplétion de chemin ; résultats plus précis, moins de tokens consommés
- Règle mnémotechnique : `!` pour les commandes que vous voulez que Claude voie, `@` pour cibler un fichier précis sans gaspiller de tokens

Tip 8 — Démarrez depuis la racine

- Lancer Claude Code depuis le **dossier racine** du projet lui donne un aperçu global de l'architecture, une meilleure compréhension de la structure (backend, frontend, config, scripts), et l'accès aux fichiers de configuration (`.env`, `tsconfig.json`, `docker-compose.yml`...)
- Pour aller plus loin, spécifier des répertoires supplémentaires dès le lancement avec `--add-dir` : `claude --add-dir ../libs ../shared`

Tip 9 — `/init` pour votre premier CLAUDE.md

- `/init` analyse la codebase (architecture, stack, conventions) et génère un `CLAUDE.md` adapté, relu à chaque session
- Deux portées : `.claude/CLAUDE.md` → local au projet ; `~/.claude/CLAUDE.md` → global, tous projets
- À utiliser comme point de départ, puis affiner manuellement. Ne jamais partir de zéro.

Tip 10 — Combinez CLAUDE.md et mémoire automatique

Deux systèmes complémentaires, tous deux chargés au début de chaque conversation :

- **CLAUDE.md** (géré par vous) : court et précis (300 lignes max), indiquez ce qu'il *ne faut pas* faire, mettez-le à jour avec les décisions techniques
- **Mémoire automatique** (gérée par Claude) : se construit via vos corrections, ne duplique pas le contenu de `CLAUDE.md`

Dans les deux cas, `/memory` permet de consulter et modifier ce que Claude a mémorisé sur le projet.

Tip 11 — Maîtrisez la gestion du contexte

Le contexte s'accumule progressivement au fil d'une session (messages, fichiers lus, résultats de commandes) dans une fenêtre de tokens limitée par le modèle (Sonnet 4.6, Opus 4.6...). Passé un certain point, Claude peut « oublier » des instructions antérieures et produire des réponses incohérentes.

- `/compact` compresse l'historique et libère de l'espace sans repartir de zéro (option : `/compact garde l'API développée et supprime les explorations`)

Tip 12 — Utilisez `/clear` à chaque changement de tâche

- `/clear` efface l'historique de conversation et repart de zéro — équivalent à recréer une nouvelle session
- Évite les compactions coûteuses sur du contexte devenu inutile → sessions plus rapides, moins chères, moins de réponses biaisées par un contexte obsolète

Tip 13 — Utilisez `/context` pour diagnostiquer

- Affiche une représentation visuelle (grille colorée) de l'utilisation du contexte, avec la répartition des éléments chargés (fichiers, instructions, historique)
- Permet de comprendre ce que Claude « voit » réellement, d'identifier les éléments manquants ou superflus, et d'éviter la surcharge de contexte

Tip 14 — Structurez vos prompts efficacement

6 principes issus des équipes Anthropic :

Structure : (1) être explicite sur le **résultat attendu**, (2) fournir le contexte et le **pourquoi**, (3) **découper en étapes** (step-by-step)

Précision : (4) inclure des **exemples** du résultat voulu, (5) **formuler en positif** (quoi faire, pas quoi éviter), (6) définir un **critère de validation** (tests, outputs)

Astuce bonus : pour les longs prompts, dicter avec `/voice` (puis `<Space>` pour commencer) ou l'app **SuperWhisper** sur macOS.

Tip 15 — Commencez d'abord en mode Plan

- `Shift+Tab` pour switcher entre les modes
- Workflow recommandé : démarrer en **mode Plan**, challenger les décisions de Claude, itérer, ne jamais accepter le premier plan — l'écriture du code (mode Accept) vient en dernier et c'est la partie la plus facile

Tip 16 — Utilisez un modèle IA adapté à votre besoin

Changer de modèle avec `/model` en mode interactif, ou via les alias supportés (`default|sonnet|opus|haiku|sonnet[1m]|opusplan`) :

- **Haïku** → tâches simples, rapide et économique
- **Sonnet** → usage quotidien, bon équilibre
- **Sonnet 1M** → gros fichiers / longues sessions
- **Opus** → problèmes complexes, raisonnement avancé

L'alias `/model opusplan` est "intelligent" : il bascule automatiquement d'Opus vers Sonnet selon la nature de la tâche.

Tip 17 — Ajustez l'effort du modèle avec `/effort`

Contrôle la profondeur de raisonnement du modèle :

- `low` → tâches simples, faible latence, moins de tokens
- `medium` → optimisation des coûts
- `high` → bon équilibre coût / raisonnement
- `xhigh` → recommandé pour les tâches complexes
- `max` → raisonnement maximal (surcoût +++) — s'applique uniquement à la session courante et requiert Opus 4.6

`low`, `medium`, `high` et `xhigh` persistent entre sessions. Sinon, ajouter le mot **`ultrathink`** dans un prompt pour un raisonnement plus poussé sans changer le niveau d'effort global.

Tip 18 — Maîtrisez Escape et Double Escape

- `Escape` pendant une réflexion en cours → interrompt Claude proprement sans quitter la session
- Input non vide + `Esc+Esc` → efface le prompt en cours de frappe
- Input vide + `Esc+Esc` → affiche la liste des messages précédents
- `/rewind` pour rembobiner la conversation et/ou le code à un point antérieur ; `/keybindings` pour personnaliser les raccourcis

Tip 19 — Glissez vos screenshots dans Claude Code

Un screenshot vaut mille explications : Claude peut analyser une UI, des erreurs visuelles, des logs affichés, la structure d'une page…

- Spécifier le chemin : `/path/to/image.png`
- Ou drag-and-drop direct depuis le Finder / l'Explorateur
- Sur macOS : `Cmd+Shift+Ctrl+4` capture une zone dans le presse-papiers, puis `Ctrl+V` pour coller directement dans Claude

Tip 20 — Utilisez `/fork` pour tenter une autre approche

- `/fork` (alias de `/branch`) crée une copie de la session courante pour explorer une approche différente, sans toucher à la session principale
- Idéal pour : hésiter entre deux stratégies d'architecture, tester une approche risquée, comparer deux implémentations en parallèle
- `/resume` pour revenir sur la branche initiale

Tip 21 — Posez des questions hors du contexte avec `/btw`

- `/btw` crée une question **parallèle** sans polluer le contexte principal
- Exemple : pendant que Claude refactore un module TypeScript, `/btw c'est quoi ce pattern en JS ?` — la réponse arrive sans interrompre ni contaminer le flux de travail en cours

Tip 22 — Inspectez et maîtrisez les sorties

Quatre commandes pour garder la main sur l'output de Claude :

- `/diff` — ouvre un viewer interactif (←/→) pour naviguer entre le git diff global et les diffs à chaque étape de Claude
- `/copy` — copie la dernière réponse dans le presse-papiers ; `/copy n` pour copier les *n* dernières réponses
- `/export` — ouvre une boîte de dialogue pour copier ou enregistrer dans un fichier ; `/export ma_session.md` pour écrire directement dans un fichier
- `/undo` — annule la dernière modification effectuée par Claude

Tip 23 — Qualité continue avec `/review` et `/simplify`

- `/review` (local) ou `/ultrareview` (cloud) — génère un retour structuré dans le terminal : identifie les bugs et edge cases potentiels, signale les manques de documentation ou de clarté, détecte les risques de performance
- `/simplify` — lance trois agents d'analyse en parallèle, applique directement les corrections suggérées ; on peut cibler un aspect précis : `/simplify focus sur la lisibilité du code`

Tip 24 — `/insights` pour analyser vos habitudes

`/insights` génère un rapport analysant vos sessions Claude Code : il décortique votre usage réel (style de dev assisté par l'IA), pointe vos pertes de temps, propose des optimisations concrètes et aide à automatiser encore plus de workflows. Particulièrement précieux après plusieurs semaines ou mois d'utilisation — il montre non seulement ce que vous faites, mais surtout ce que vous pourriez améliorer.

Tip 25 — Automatisez des actions avec les `/hooks`

Les hooks sont des scripts déclenchés automatiquement à des moments clés du workflow Claude Code. Ils permettent de gérer des actions déterministes (formatage, tests, sécurité, logging…) et supportent les tâches asynchrones.

Événements disponibles : `PreToolUse` (avant chaque action de Claude), `PostToolUse` (après chaque action), `Notification` (sur notification), `Stop` (quand Claude a terminé), `FileChanged` (sur changement d'un fichier)…

Tip 26 — Je n'ai pas eu le temps de le noter

Tip 27 — Automatisez avec `/loop` et `/schedule`

Deux outils complémentaires pour les tâches répétitives :

- `/loop` **(local)** — tourne dans le terminal actuel ; idéal pour surveiller un déploiement, une PR ou relancer une vérification ; s'arrête à la fermeture du terminal. Exemple : `/loop 5e vérifie si le déploiement est terminé`
- `/schedule` **(cloud)** — tourne sur l'infrastructure Anthropic, sans votre machine ; déclenché par un calendrier, une API ou des événements GitHub ; continue même si votre ordinateur est éteint

Tip 28 — Créez vos propres skills !

Les skills étendent les capacités de Claude Code avec des commandes métier sur mesure. Procédure en 4 étapes (exemple : `summarize-text`) :

1. `mkdir -p ~/.claude/skills/summarize-text`
2. `touch ~/.claude/skills/summarize-text/SKILL.md`
3. Renseigner le nom, la description et les instructions dans ce fichier
4. Tester de deux façons : en langage naturel (`Donne moi un résumé de ce long texte: …`) ou via la slash command (`/summarize-text [Pasted 23 lines]`)

Doctolib crée ses propres skills adaptés à sa codebase et ses besoins métier — une pratique fortement encouragée.

Tip 29 — `/mcp` : soyez sélectif !

Les MCP (Model Context Protocol) sont des serveurs externes que Claude appelle comme des outils — hébergés par les vendors, accessibles via une simple URL.

MCP incontournables : **GitHub** (PRs, Issues, Actions), **Jira/Confluence** (tickets, docs), **Sentry** (erreurs prod, debug), **Datadog** (logs, métriques, traces), **Context7** (docs à jour en contexte).

Bonne pratique critique : n'activer que ce dont on a **vraiment besoin**. 15 MCP actifs en permanence = contexte saturé. Utiliser `/mcp` pour voir les serveurs actifs et désactiver ceux qui ne sont pas nécessaires à la tâche en cours.

**Tip 30 — Blague de l'orateur 😃 **

Fin de session dans la bonne humeur.

Vendredi 24 avril

10h30 — Limits, Requests, QoS, PriorityClasses : démystifions le scheduling dans K8s !

Orateurs : Denis Germain (blog.zwindler.fr) et **Quentin Joly** (une-tasse-de.cafe) — tous deux ingénieurs chez **Lucca**. Talk particulièrement pédagogue, illustré par une série de démos live poussant les clusters dans leurs derniers retranchements.

Problématique de départ : limits, requests, QoS, PriorityClasses — tout le monde sait qu'il faut les spécifier en production, mais peu comprennent vraiment ce qu'il se passe sous le capot quand les ressources viennent à manquer.

Partie 1 — Limits et Requests : les bases

Deux concepts distincts à ne pas confondre :

- **Limits** : la contrainte de ressources que le Pod ne peut *pas* dépasser
- **Requests** : ce que le Pod *demande* au scheduler pour être placé sur un nœud

Les ressources se divisent en deux familles :

- **Compressibles (CPU)** : on peut partager les cycles à l'infini — le pod sera juste ralenti. 100m (millicores) = 10 % d'un CPU, 1000m = un cœur complet.
- **Incompressibles (RAM, disque)** : on ne peut pas partager un octet de RAM. Si on dépasse, le pod est tué → `OOMKilled`.

Ce qu'il se passe sous le capot avec les CPU limits — le throttling

Quand on configure `cpu: 300m` en limits, Kubernetes alloue en réalité **30 ms de CPU toutes les 100 ms** via les cgroups Linux. Si le conteneur consomme ses 30 ms avant la fin de la fenêtre (ex. 20 ms + 20 ms + 10 ms = 50 ms sur deux requêtes bursts), il est mis en **pause forcée** (*throttling*) jusqu'à la prochaine fenêtre — même si des CPU sont disponibles sur le nœud.

Conséquence directe : **mettre des limits CPU peut exploser les performances d'un frontend web** en introduisant des latences artificielles. La recommandation est de **ne pas mettre de limits CPU** sur les charges sensibles à la latence (sauf exception justifiée).

Partie 2 — Requests et scheduling

Sans requests, le scheduler n'a aucun moyen de savoir quelle charge peut tenir sur quel nœud — comme essayer de répartir des colis dans des camions sans connaître leur poids. Bien spécifier les requests permet au scheduler de placer les pods sur les bons nœuds et d'éviter qu'un pod reste `Pending` faute de place.

Les `requests.cpu` sont traduites en **cpu.shares** au niveau des cgroups : un pod à 100m reçoit 4 shares, un pod à 500m en reçoit 20. Ces shares servent à arbitrer l'accès au CPU *uniquement quand il y a contention* — hors contention, un pod peut consommer bien plus que sa request. Mettre des requests anormalement basses pour « s'assurer d'être schedulé » aboutit à du sur-provisionnement et des comportements imprévisibles sous charge.

Partie 3 — QoS (Quality of Service)

Kubernetes dérive automatiquement une classe QoS de la configuration requests/limits :

QoS	Condition	Comportement en cas de pression mémoire
Guaranteed	limits = requests pour CPU et RAM	Dernier à être évincé
Burstable	limits > requests (ou limits seules)	Évincé selon dépassement
BestEffort	Aucune limits ni requests	Premier à être évincé

La QoS est le mécanisme de **survie en cas de surcharge inattendue** : quand un nœud manque de mémoire, Kubernetes évince d'abord les BestEffort, puis les Burstable qui dépassent leurs requests, et protège les Guaranteed.

Partie 4 — Priority Classes

La QoS gère la survie en cas de crise, mais ne permet pas de garantir le déploiement d'applications critiques sur un cluster saturé. Les **PriorityClasses** comblent ce besoin : elles permettent de préempter des pods de faible priorité pour faire de la place à un pod critique, même quand le cluster est plein.

Takeaways

- **Pas de limits CPU** (sauf exception) : le throttling introduit des latences artificielles et nuit à la performance
- **Requests et limits réalistes partout** : le scheduler en a besoin pour travailler correctement, et des valeurs sous-estimées causent des surprises sous charge
- **QoS** → survie en cas de surcharge inattendue
- **PriorityClasses** → assurer le déploiement d'applications critiques quand le cluster est sous pression

11h30 — Et si votre backend savait parler ? Créer des agents vocaux IA en temps réel (Amazon Web Services)

Orateurs : Mehdi Oualiken et Abdessamii Lazghab (Amazon Web Services).

Atelier pratique de 2 heures centré sur **Amazon Nova Sonic**, le modèle speech-to-speech d'AWS : construction d'un agent conversationnel vocal de bout en bout avec capture audio en streaming, compréhension du langage, logique de dialogue et génération de réponses parlées. Format 100 % hands-on, pré-requis : bases Python et APIs.

> *(Note personnelle)* Je suis arrivé en cours de route faute de places dans d'autres sessions — je n'ai donc qu'une partie du contenu.

Ce que j'en ai retenu :

- Beaucoup de code **Python**
- Mise en place de **guardrails** pour éviter les réponses inappropriées de l'agent vocal
- Le pipeline adresse les contraintes spécifiques du temps réel : latence, streaming, gestion du multi-tours

Strands Agents — l'orchestrateur au cœur du lab

Le framework utilisé pour orchestrer le pipeline voix-à-voix est **Strands Agents**, SDK open source publié par AWS en mai 2025. Son principe : une approche **model-driven** où c'est le LLM lui-même qui pilote le raisonnement et le choix des outils, sans que le développeur ait à écrire de logique d'orchestration complexe. On définit trois composants — un modèle, un prompt système, une liste d'outils — et l'agent gère le reste.

Issu de l'expérience interne d'AWS sur Amazon Q Developer, Strands a permis de réduire les délais de mise en production d'agents de plusieurs mois à quelques jours ou semaines. Compatible Amazon Bedrock, Anthropic, OpenAI, Gemini et Ollama, il supporte nativement le protocole MCP et le streaming audio bidirectionnel pour les agents vocaux.

13h30 — Migration quantique : *Theory to Practice — Real-World Lessons in Post-Quantum Cryptography Migration*

Présenté par Akihiro Nishikawa (Cloud Solution Architect, Microsoft).

Pourquoi agir maintenant ?

- Aucun algorithme cryptographique ne reste sûr éternellement (DES, SHA-1 déjà cassés ; RSA et ECDSA suivront)
- Attaque **HNDL** (*Harvest Now, Decrypt Later*) : des acteurs capturent dès aujourd'hui des données chiffrées pour les déchiffrer plus tard avec un ordinateur quantique
- Les estimations pour casser RSA-2048 chutent : 20 M qubits bruyants en 2019 → ~1 M en 2025 → ~10-100 k en mars 2026
- **65 %** des responsables sécurité sont préoccupés par le HNDL, **69 %** pensent que le chiffrement actuel sera cassé dans 5 ans, mais seulement **5 %** ont déployé du chiffrement post-quantique
- Objectif de transition PQC fixé à **~2035** par les USA, l'UE, le UK, le Japon (~2030 pour l'Australie)

Standards NIST PQC : FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), FIPS 205 (SLH-DSA) ; FN-DSA et HQC en cours de standardisation.

Support Java :

- Java 21 : KEM API → Java 24 : ML-KEM, ML-DSA → Java 25 : KDF API → **Java 27 (sept. 2026) : TLS 1.3 hybride (JEP 527)**
- JEP 527 : `X25519MLKEM768` activé par défaut, le handshake passe en hybride si le pair le supporte, sans changement de code applicatif
- En attendant : utiliser **Bouncy Castle**

Méthode de migration en 8 étapes :

1. **Inventaire** en 4 couches : dépendances, code source/frameworks, configuration/KeyStore, runtime (JFR) — « *You cannot migrate what you cannot find* »
2. **Priorisation** : transit d'abord, données chiffrées longue durée ensuite, signature de code en dernier
3. **Hybride** : classique + PQC pendant la transition (pur PQ uniquement en écosystème contrôlé)
4. **Crypto-agility** : ne jamais coder en dur les algorithmes, externaliser les choix dans la configuration, pattern Strategy
5. **Approche phasée** : outside-in (edge d'abord) ou inside-out (trust core d'abord) ; KEM d'abord, authentification après
6. **Performance** : mesurer CPU/mémoire par handshake, latence p50/p95, taux de réutilisation de connexion
7. **Tests** : interopérabilité, performance, fallback/rollback, enforcement de politique
8. **Timeline** : cas client anonymisé — 200+ services, 3 000+ points de contact, majorité sur Java 17, migration sur plusieurs années

HNDL cible l'échange de clés, pas l'authentification : casser l'échange = déchiffrer toutes les sessions passées ; casser la signature = usurper de futures sessions uniquement. D'où la priorité au KEM.

À retenir : *« Inventory first. Hybrid key exchange first. Signatures next. »*

14h35 — Histoire des noms de domaine

Anatomie d'un nom de domaine :

- Étiquette
- Extension
- Extensions de second niveau possibles : `gouv.fr`
- TLD (`.fr`) — SLD (`.gouv.fr`) — 3LD (`anjo.aichi.jp`)

Types d'extensions :

- **ccTLD** (country code) / **gTLD** (general)
- **IDN** (International Domain Name) depuis 2003

Architecture DNS :

- **ICANN** à la racine
- **Whois** : déprécié en 2025 (pas de standard pour les clés et contenus)
- **RDAP** : nouveau protocole, JSON via HTTPS

15h40 — Refactorisation sans tout casser : anatomie des patterns de modernisation incrémentale

Présenté par Hela Ben Khalfallah (*Crafting Clean Code*).

Constat de départ : tout le monde veut moderniser un legacy, la tentation est de tout réécrire d'un coup, le résultat habituel est régressions, blocages, rollback. La modernisation est un problème de **séquençage**, pas de remplacement total.

Loi de Lehman (1974) : l'entropie d'un système logiciel augmente naturellement avec le temps ($E(t) = E_0 + \lambda t$). Ne rien faire, c'est déjà reculer. La maintenance incrémentale compense cette croissance par des corrections régulières.

Règle fondamentale : un seul axe de changement à la fois. La disruption se multiplie, pas s'additionne ($d(1 \text{ module}) \approx 0.05$ vs $d(60 \text{ modules}) \approx 0.95$).

3 coupes chirurgicales — 3 patterns :

- **Axe 1 — Le contrat** (API, schéma, payload) → **Parallel Change** (*Expand / Migrate / Contract*). Exemple détaillé : migration d'un champ `is_completed: boolean` vers `status: enum` dans un schéma bancaire PSD2. Dual-write, backfill, puis suppression quand la télémétrie prouve un usage ancien à zéro. 4 déploiements, zéro downtime.
- **Axe 2 — La brique interne** (lib, framework appelé partout en interne) → **Branch by Abstraction** (*Abstract / Migrate / Build / Switch / Cleanup*). Exemple détaillé : remplacement d'une lib UI vendor (@vendor/ui, 1 200 imports dans 66 apps) par une lib interne. Façade avec feature flags par composant,

migration composant par composant, rollback par toggle. Résultat : 45 kB Emotion runtime → 2 kB CSS natif, accessibilité WCAG AA, charte maison.
- **Axe 3 — La topologie** (découpage, déploiement) → **Strangler Fig** (*Shell / Extract / Reroute / Repeat / Remove*). Exemple détaillé : extraction d'une SPA monolithique (4,2 Mo, build 8 min, 3 équipes couplées) en micro-frontends via Module Federation. Chaque extraction = 1 ligne dans la table de routage du shell, rollback = supprimer la ligne. Résultat : ~200 kB et ~15 sec de build par module.

Plomberie sous-jacente : les 3 patterns s'appuient sur 4 design patterns combinés — Facade (simplifier), Adapter (traduire), Proxy (aiguiller), Mediator (coordonner).

Cas réels : Netflix Cosmos (Strangler Fig, de Reloaded à Cosmos en 3 couches), GitHub Scientist (Branch by Abstraction, les deux implémentations tournent en parallèle en prod, divergences loguées, bascule quand mismatch = 0).

3 questions avant de toucher au code :

1. **Qu'est-ce qui bouge, qu'est-ce qui reste ?** → détermine le pattern
2. **Comment on revient en arrière demain matin ?** → sans plan de rollback, on ne commence pas
3. **Comment on sait que ça marche sans demander à personne ?** → on ne migre pas ce qu'on ne mesure pas

Règle des changements : Réversibles, Observables, Incrémentaux.

> *« L'incrémental n'est pas de la prudence, c'est de l'ingénierie. » — « First make the change easy, then make the easy change. »* (Kent Beck)

17h — De la galère des setups locaux aux DevContainers : notre retour d'expérience

Orateur : Thomas Rumas (ADEO). Tool in Action présentant l'adoption des DevContainers pour unifier et automatiser les environnements de développement chez ADEO. Le point de départ : les scripts bash atteignent leurs limites dès qu'il s'agit de standardiser des environnements à grande échelle.

Problème de départ : l'évaluation de skills IA qui triche

Thomas Rumas raconte comment, en développant des skills GitHub Copilot chez ADEO, ses évaluations donnaient des résultats trop bons. Raison : le modèle scannait l'environnement local (repos, documentation…) au-delà du périmètre prévu, faussant complètement les métriques. La solution : un environnement isolé et contrôlé — un DevContainer — dans lequel l'IA n'a accès qu'à ce qu'on lui donne explicitement.

Deux problèmes, une même solution

- **Évaluation de skills** : contexte strictement contrôlé, résultats fiables
- **Expérimentation sans risque** : tester Claude Code, OpenCode, un nouveau skill communautaire sans risquer de casser son environnement local ; le container est jetable et reproductible

Le projet `devcontainer-ai` (open source)

Thomas Rumas a publié devcontainer-ai, un DevContainer clé en main pour travailler avec les agents IA :

Socle toujours présent : image base Ubuntu 24.04 Microsoft, Node.js (via nvm), Python 3.12, Docker-in-Docker, GitHub CLI, Git LFS. Choix délibéré de Node.js et Python : la quasi-totalité de l'écosystème agent tourne sur l'un ou l'autre.

Outils IA configurables via un fichier `devcontainer.env` (git-ignoré) :

Outil	Rôle
GitHub Copilot CLI	Intégration Copilot dans le terminal
Claude Code	Agent de coding autonome Anthropic
OpenCode	Agent de coding alternatif
Playwright CLI	Navigation web pour les agents

Cycle de vie en deux phases :

- `postCreateCommand` (build) : lit `devcontainer.env`, installe les outils sélectionnés
- `postStartCommand` (chaque démarrage) : lit `skills.txt`, installe les skills déclarés — sans rebuild

```
# .devcontainer/skills.txt
npx skills add https://github.com/anthropics/skills --skill skill-creator
npx skills add https://github.com/microsoft/playwright-cli --skill playwright-cli
```

Les skills s'installent dans `.agents/skills/` (universel) et `.claude/skills/` si Claude Code est détecté.

Démarrage en trois commandes

```
git clone https://github.com/ThomasRumas/devcontainer-ai
cp .devcontainer/devcontainer.env.example .devcontainer/devcontainer.env
code .  # → "Dev Containers: Reopen in Container"
```

Un script de vérification (`.devcontainer/tests/verify.sh`) confirme que tout est correctement installé après le build.

Messages clés

- Les DevContainers éliminent les problèmes de setup local et permettent un onboarding en un clic
- Préférer les bases images **Microsoft** (`mcr.microsoft.com/devcontainers/…`) pour la fiabilité et la maintenance
- Travail à distance possible avec **VSCode** via l'extension Dev Containers
- Le container est aussi un **bac à sable pour agents IA** : le blast radius reste contenu, l'environnement hôte reste propre

---

Synthèse

Trois enseignements se dégagent de cette édition, qui se répondent les uns aux autres.

1. L'IA agentique a basculé dans l'industrialisation, mais l'enjeu réel est ailleurs. Le code ne suffit plus : ce qui fait la différence, c'est le contexte versionné, les specs exécutables, la discipline (`AGENTS.md` ≤ 200 lignes, contexte ≤ 80 %, une session = une tâche). Doctolib, WeScale, Septeo : les retours d'expérience convergent

vers une même conclusion — le développeur devient orchestrateur. Une transition que les joueurs d'échecs ont vécue il y a vingt ans face aux moteurs, et que les créatifs vivent en ce moment même : Martignole et Itschner l'ont rappelé sans détour — *« ce que vivent les créatifs aujourd'hui préfigure ce que vivront les développeurs demain »*.

2. Les fondamentaux ne disparaissent pas — ils prennent du poids. Quand l'IA produit du code plus vite que jamais, la dette de conception s'accumule plus vite, l'observabilité devient plus critique, la maîtrise du temps et du scheduling moins négociable. Hive Pattern, OpenTelemetry chez Winamax, NTP/PTP, limits/requests Kubernetes, refactorisation incrémentale : autant de sujets qui semblent triviaux jusqu'à l'incident de production, et qui prennent une importance redoublée dans un monde où le code arrive plus vite à la rampe.

3. La sécurité et la responsabilité passent au premier plan. DevSecOps mature avec *shift-left* systématique chez Piguet Galland, anticipation post-quantique face à la menace HNDL, et — au-delà du purement technique — la conférence d'Estelle Landry sur Pix Junior, qui interroge directement le rôle des bâtisseurs de la tech vis-à-vis des plus jeunes. La rigueur d'ingénierie ne s'arrête pas aux portes du SI.

Pour boucler la métaphore d'ouverture : si l'IA a effectivement augmenté les joueurs d'échecs sans les remplacer, c'est parce que ceux-ci ont accepté de retravailler leur préparation, leur méthode, leur lecture du jeu. Devoxx 2026 envoie le même signal aux ingénieurs — **l'augmentation par l'IA est un contrat qui exige plus de discipline, pas moins, et qui s'accompagne d'une responsabilité élargie**.